```
import numpy as np

class GridWorldMDP:
    def __init__(self, size, goal, trap):
        self.size = size
        self.goal = goal
        self.trap = trap
        self.states = [(i, j) for i in range(size) for j in range(size)]
        self.actions = ['UP', 'DOWN', 'LEFT', 'RIGHT']

    def get_next_state(self, state, action):
        i, j = state

        if action == 'UP':
            i -= 1
        elif action == 'DOWN':
            i += 1
        elif action == 'LEFT':
            j -= 1
        elif action == 'RIGHT':
            j += 1

        i = max(0, min(i, self.size - 1))
        j = max(0, min(j, self.size - 1))

        return (i, j)

    def get_reward(self, state):
        if state == self.goal:
            return 0
        elif state == self.trap:
            return -10
        else:
            return -1


def policy_iteration(mdp, gamma=0.9, theta=0.01):

    # Initialize random policy
    policy = {
        s: np.random.choice(mdp.actions)
        for s in mdp.states
        if s != mdp.goal and s != mdp.trap
    }

    V = {s: 0 for s in mdp.states}

    while True:

        # -------- Policy Evaluation --------
        while True:
            delta = 0
            for state in mdp.states:
                if state == mdp.goal or state == mdp.trap:
                    continue

                v = V[state]
                action = policy[state]

                next_state = mdp.get_next_state(state, action)
                reward = mdp.get_reward(next_state)
```

```
                V[state] = reward + gamma * V[next_state]

                delta = max(delta, abs(v - V[state]))

            if delta < theta:
                break

        # -------- Policy Improvement --------
        policy_stable = True

        for state in mdp.states:
            if state == mdp.goal or state == mdp.trap:
                continue

            old_action = policy[state]

            best_action = None
            best_value = float('-inf')

            for action in mdp.actions:
                next_state = mdp.get_next_state(state, action)
                reward = mdp.get_reward(next_state)

                value = reward + gamma * V[next_state]

                if value > best_value:
                    best_value = value
                    best_action = action

            policy[state] = best_action

            if old_action != best_action:
                policy_stable = False

        if policy_stable:
            break

    return policy, V


# MAIN
if __name__ == "__main__":
    mdp = GridWorldMDP(size=3, goal=(2, 2), trap=(1, 1))

    policy, V = policy_iteration(mdp)

    print("Value Function:")
    for s in V:
        print(s, ":", round(V[s], 2))

    print("\nOptimal Policy:")
    for s in policy:
        print(s, "->", policy[s])
```

```
Value Function:
(0, 0) : -2.71
(0, 1) : -1.9
(0, 2) : -1.0
(1, 0) : -1.9
(1, 1) : 0
(1, 2) : 0.0
(2, 0) : -1.0
(2, 1) : 0.0
(2, 2) : 0
```

```
Optimal Policy:
(0, 0) -> DOWN
(0, 1) -> RIGHT
(0, 2) -> DOWN
(1, 0) -> DOWN
(1, 2) -> DOWN
(2, 0) -> RIGHT
(2, 1) -> RIGHT
```